

# Kiste v0.9.2 Spec

Expanded Doctor: Lint, Security, Performance, Custom Scripts, Repo/Workspace Validation

**One-sentence spec: Kiste v0.9.2 expands** `kiste doctor <path>` **into the standard validation engine for repo/workspace contract checks, lint, security, performance-readiness, integration readiness, and explicitly configured custom scripts - while keeping the CLI/API consistent with the current Kiste model.**

## 1. Design Decision

Kiste should not add separate top-level commands such as `kiste test`, `kiste lint`, `kiste security`, or `kiste perf` in v0.9.2. Instead, these capabilities are folded into `kiste doctor` as check groups. This keeps the surface area small and makes doctor the trusted quality gate.

## 2. Canonical CLI Contract

```
kiste doctor <path>
kiste doctor <path> --scope repo
kiste doctor <path> --scope workspace
kiste doctor <path> --profile safe
kiste doctor <path> --profile default
kiste doctor <path> --profile strict
kiste doctor <path> --profile release
kiste doctor <path> --group lint
kiste doctor <path> --group security
kiste doctor <path> --group performance
kiste doctor <path> --group custom
kiste doctor <path> --json
```

The path is required or defaults to the current directory. Scope is only used when auto-detection is ambiguous. Kiste keeps `repo` and `workspace` because they describe real target shapes. There is no `self` scope; the Kiste repo is just a repo with Kiste-specific metadata.

| Concept | Meaning                            | Examples                            |
|---------|------------------------------------|-------------------------------------|
| scope   | Shape of target                    | repo, workspace                     |
| profile | Depth and strictness of validation | safe, default, strict, release      |
| group   | Area of checks to run              | lint, security, performance, custom |
| json    | Machine-readable CI output         | DoctorReport JSON                   |

## 3. Auto-detection Rule

```
if kiste.workspace.yaml exists:
    scope = "workspace"
elif kiste.yaml exists:
    scope = "repo"
else:
    infer from files and default to "repo"
```

If the path points to the Kiste repo, Kiste detects that from `kiste.yaml` metadata and enables Kiste-specific architecture/module/plugin checks through the selected profile. No special self flag is needed.

## 4. Profiles

| Profile | Purpose | Default behavior |
|---------|---------|------------------|
|---------|---------|------------------|

|         |                                    |                                                                                 |
|---------|------------------------------------|---------------------------------------------------------------------------------|
| safe    | Untrusted repo or quick inspection | No custom scripts, no external execution, read-only scans only                  |
| default | Normal local diagnosis             | Config, repo/workspace, lint readiness, security basics, performance readiness  |
| strict  | CI validation                      | Fails on serious contract/security/lint/performance issues                      |
| release | Pre-release quality gate           | Strict plus integration readiness, custom scripts if configured, release checks |

## 5. Check Groups

Doctor groups let users run a precise subset without adding new top-level commands.

```

config
repo
workspace
architecture
module
plugin
bootstrap
target
gitops
argocd
cloud
data
blob
integration
lint
security
performance
custom
release

```

## 6. Lint Group

The lint group checks static quality and project hygiene. Kiste itself can be Python-first, but doctor must be repo-first and adapter-based.

| Runtime         | v0.9.2 lint support                                                       |
|-----------------|---------------------------------------------------------------------------|
| Common          | YAML/JSON validation, config consistency, basic repo hygiene              |
| Python          | Ruff detection/run, mypy or pyright detection                             |
| Node/TypeScript | package.json, tsconfig, ESLint/Prettier detection, basic script discovery |
| Go              | go.mod, gofmt/go vet/staticcheck detection                                |
| Docker          | Dockerfile and .dockerignore readiness                                    |
| Java/Rust/C/C++ | Detect-only initially                                                     |

Default lint behavior is read-only. It may report findings or recommend commands, but it should not auto-fix files in v0.9.2.

## 7. Security Group

The security group detects obvious deployment and repository risks. It is not a full security audit.

```
kiste doctor <path> --group security
```

```

Checks:
- plaintext secrets and hardcoded tokens
- committed .env risk
- missing dependency lockfile
- overly broad GitHub Actions permissions
- Dockerfile root user / latest tag risks
- unsafe Kubernetes privileges

```

- missing SOPS/secret mapping
- Ansible Vault and Pulumi secret references
- large private blob exposure risk

Security reports risks and recommendations. It does not rotate secrets, delete files, rewrite workflows, or mutate infrastructure by default.

## 8. Performance Group

In v0.9.2, performance means performance-readiness, not a full benchmark suite.

```
kiste doctor <path> --group performance
```

Checks:

- large files in repo
- huge Docker build context
- missing .dockerignore
- large dependency footprint
- missing resource limits
- missing healthcheck/readiness/liveness probe
- unbounded worker settings
- slow startup risk
- PyTorch CUDA/runtime mismatch
- large model/blob download risk

Real benchmark execution should remain optional and explicit in later versions, for example `--run-benchmarks --confirm`.

## 9. Custom Scripts

Custom scripts allow repo owners to connect their existing test/build commands to doctor without making Kiste replace pytest, npm test, go test, or other native frameworks.

```
doctor:
  custom:
    enabled: true
    scripts:
      - name: unit-tests
        command: pytest
        args: ["tests/unit"]
        timeout_seconds: 120
        required: true

      - name: frontend-tests
        command: npm
        args: ["test", "--", "--run"]
        working_dir: frontend
        timeout_seconds: 180
        required: false

      - name: build-check
        command: python
        args: ["-m", "build"]
        timeout_seconds: 120
        required: true
```

Custom scripts are disabled in safe mode and should not run in unknown repos unless explicitly configured or requested by the selected profile. They must have timeouts and should not receive secrets by default.

## 10. Folded Kiste Test

The useful parts of the earlier `kiste test` idea are folded into doctor:

| Old idea                       | v0.9.2 replacement                                        |
|--------------------------------|-----------------------------------------------------------|
| <code>kiste test repo .</code> | <code>kiste doctor . --scope repo --profile strict</code> |

|                               |                                                          |
|-------------------------------|----------------------------------------------------------|
| kiste test workspace .        | kiste doctor . --scope workspace --profile strict        |
| kiste test plugin ./plugins/x | kiste doctor ./plugins/x --group plugin --profile strict |
| kiste test integration        | kiste doctor . --group integration --profile strict      |
| kiste test snapshot           | kiste doctor . --group release --profile release         |

## 11. Kiste Repo Example

To validate Kiste itself, point doctor at the Kiste repo. No self scope is needed.

```
kiste doctor ./kiste --scope repo --profile release
```

This should run:

- config validation
- architecture boundary checks
- core / py / cli import rules
- plugin registry checks
- module graph checks
- kiste.lock validation
- lint group
- security group
- performance group
- integration detect/plan/validate checks
- custom scripts if configured by the release profile
- release readiness checks

## 12. DoctorReport Schema

```
{
  "schema": "kiste.doctor/v0.9.2",
  "status": "warn",
  "scope": "repo",
  "path": ".",
  "profile": "strict",
  "groups": ["config", "repo", "lint", "security", "performance", "custom"],
  "checks": [
    {
      "id": "lint.python.ruff",
      "group": "lint",
      "status": "pass",
      "message": "Ruff check passed"
    },
    {
      "id": "security.secrets.plaintext",
      "group": "security",
      "status": "fail",
      "message": "Possible plaintext secret found"
    },
    {
      "id": "performance.docker.no_healthcheck",
      "group": "performance",
      "status": "warn",
      "message": "Dockerfile has no HEALTHCHECK"
    },
    {
      "id": "custom.unit-tests",
      "group": "custom",
      "status": "pass",
      "message": "Custom script completed successfully",
      "duration_seconds": 12.4
    }
  ],
  "summary": {"pass": 31, "warn": 4, "fail": 1}
}
```

## 13. Output Layout

```
.kiste/
  doctor/
    doctor-report.json
    lint-report.json
    security-report.json
    performance-report.json
    custom-report.json
    integration-doctor-report.json
    release-doctor-report.json
```

## 14. API Contract

The Python API should mirror the CLI and avoid redundant booleans.

```
from kiste import doctor

# Normal repo diagnosis
report = doctor.run(".")

# Release-quality check
report = doctor.run(".", profile="release")

# Focused security check
report = doctor.run(".", groups=["security"])

# Workspace check
report = doctor.run(".", scope="workspace", profile="strict")
```

Profiles define common behavior. Groups define precise override. Scope is only repo/workspace, and path-based auto-detection should usually be enough.

## 15. Safety Rules

| Allowed by default                | Not allowed by default                         |
|-----------------------------------|------------------------------------------------|
| Read files and metadata           | Auto-fix lint issues                           |
| Validate config and lockfiles     | Run Pulumi up                                  |
| Scan secrets and dependency files | Run Ansible playbooks                          |
| Generate plans and reports        | Apply Kubernetes or sync Argo CD               |
| Write .kiste/doctor/*.json        | Download large blobs or mutate cloud resources |

## 16. Acceptance Criteria

```
kiste doctor .
kiste doctor . --json
kiste doctor . --profile strict
kiste doctor . --profile release
kiste doctor . --scope repo
kiste doctor . --scope workspace
kiste doctor . --group lint
kiste doctor . --group security
kiste doctor . --group performance
kiste doctor . --group custom
```

v0.9.2 is complete when doctor can produce a unified DoctorReport, support repo/workspace scopes, run lint/security/performance/custom groups, keep safe defaults, and validate the Kiste repo through the same command used for normal repos.

## Final Rule

**Kiste v0.9.2 makes `kiste doctor <path>` the single quality gate for contract validation, lint, security, performance-readiness, integration readiness, and custom scripts.**

**Scope stays repo/workspace. Profiles define depth. Groups define precision. No separate test/lint/security/perf top-level commands.**